

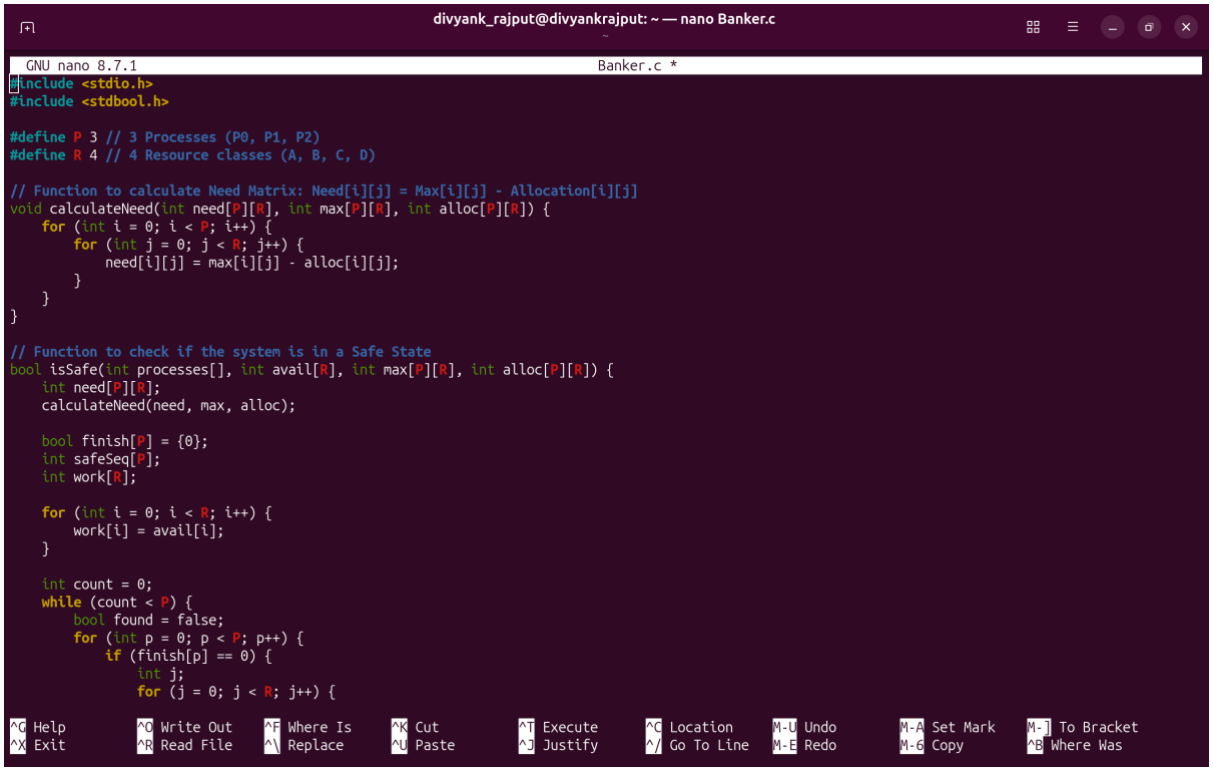
THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

Assignment 7:
Operating Systems
(MCA-103)

Divyank Rajput
2026010038
1MCA-B

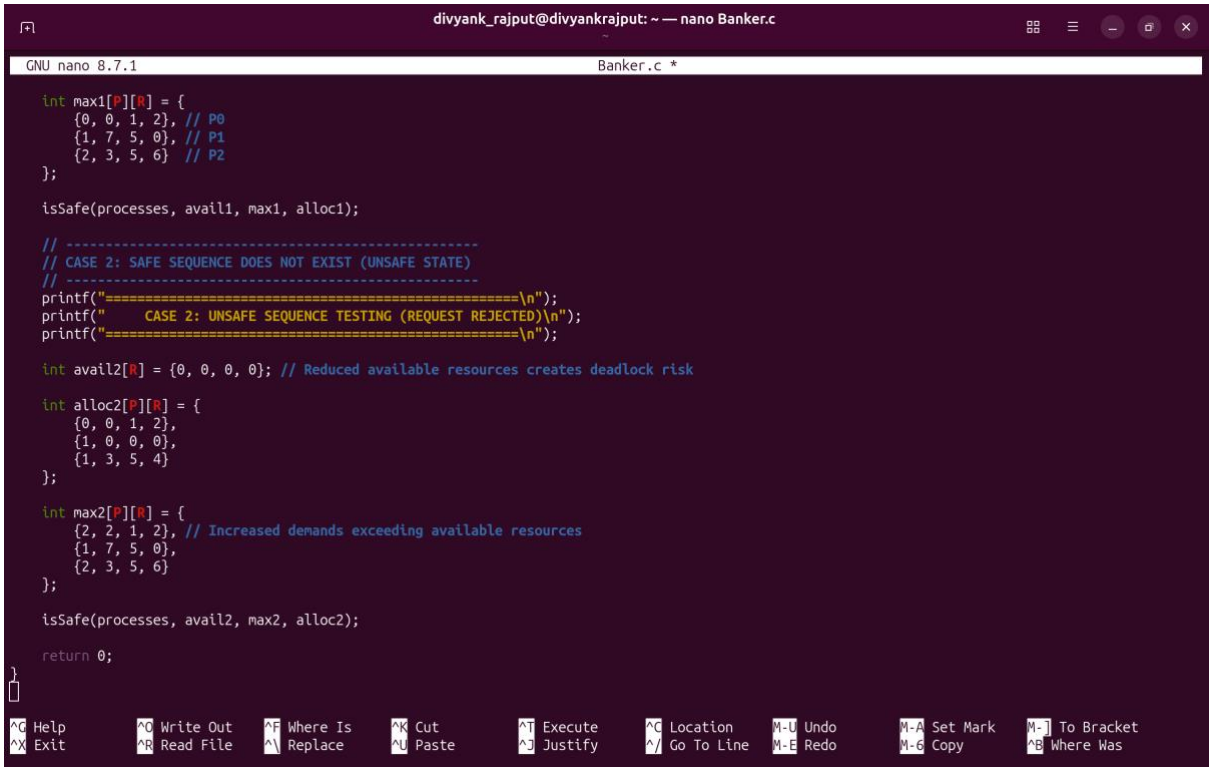
Q1. Write a program in C/C++/Java to simulate the Banker's algorithm for deadlock avoidance. Consider at least 3 processes in the system, with 4 resource classes having at least one resource instance for each class. Assume the values for Available, Allocation, MAX, and request from a particular process from your side. Program must reflect two cases, where a safe sequence exists for one and safe sequence does not exist for another.

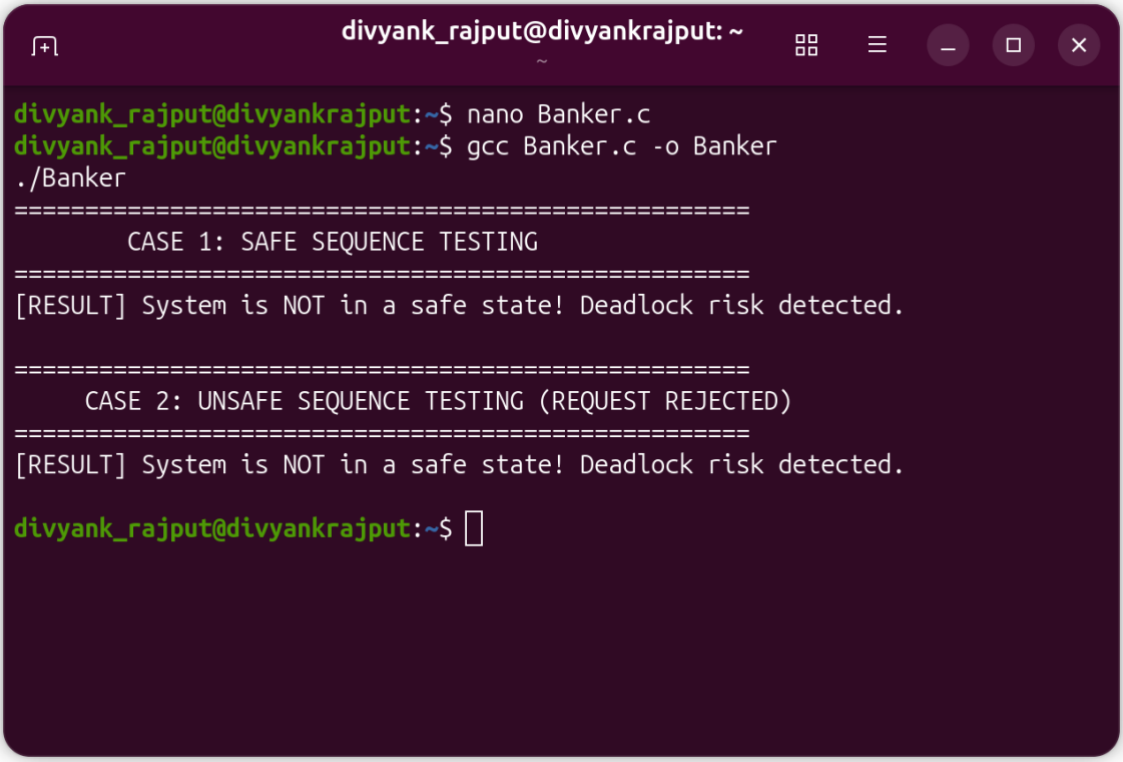
Ans:




divyank_rajput@divyankrajput: ~ — nano Banker.c
GNU nano 8.7.1 Banker.c *
int count = 0;
while (count < P) {
bool found = false;
for (int p = 0; p < P; p++) {
if (finish[p] == 0) {
int j;
for (j = 0; j < R; j++) {
if (need[p][j] > work[j])
break;
}
if (j == R) {
for (int k = 0; k < R; k++) {
work[k] += alloc[p][k];
}
safeSeq[count++] = p;
finish[p] = 1;
found = true;
}
}
}
if (found == false) {
printf("[RESULT] System is NOT in a safe state! Deadlock risk detected.\n\n");
return false;
}
}
printf("[RESULT] System is in a SAFE STATE!\nSafe Sequence: ");
for (int i = 0; i < P; i++) {
printf("P%d ", safeSeq[i]);
}
printf("\n\n");
return true;
}
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^B Where Was

divyank_rajput@divyankrajput: ~ — nano Banker.c
GNU nano 8.7.1 Banker.c *
int main() {
int processes[] = {0, 1, 2};
// --
// CASE 1: SAFE SEQUENCE EXISTS
// --
printf("==\n");
printf(" CASE 1: SAFE SEQUENCE TESTING\n");
printf("==\n");
int avail1[R] = {2, 1, 0, 0}; // Available resources (A, B, C, D)
int alloc1[P][R] = {
{0, 0, 1, 2}, // P0
{1, 0, 0, 0}, // P1
{1, 3, 5, 4} // P2
};
int max1[P][R] = {
{0, 0, 1, 2}, // P0
{1, 7, 5, 0}, // P1
{2, 3, 5, 6} // P2
};
isSafe(processes, avail1, max1, alloc1);
// --
// CASE 2: SAFE SEQUENCE DOES NOT EXIST (UNSAFE STATE)
// --
printf("==\n");
printf(" CASE 2: UNSAFE SEQUENCE TESTING (REQUEST REJECTED)\n");
printf("==\n");
int avail2[R] = {0, 0, 0, 0}; // Reduced available resources creates deadlock risk
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^B Where Was


divyank_rajput@divyankrajput: ~ — nano Banker.c
GNU nano 8.7.1 Banker.c *
int max1[P][R] = {
{0, 0, 1, 2}, // P0
{1, 7, 5, 0}, // P1
{2, 3, 5, 6} // P2
};
isSafe(processes, avail1, max1, alloc1);
// ---
// CASE 2: SAFE SEQUENCE DOES NOT EXIST (UNSAFE STATE)
// ---
printf("===\n");
printf(" CASE 2: UNSAFE SEQUENCE TESTING (REQUEST REJECTED)\n");
printf("===\n");
int avail2[R] = {0, 0, 0, 0}; // Reduced available resources creates deadlock risk
int alloc2[P][R] = {
{0, 0, 1, 2},
{1, 0, 0, 0},
{1, 3, 5, 4}
};
int max2[P][R] = {
{2, 2, 1, 2}, // Increased demands exceeding available resources
{1, 7, 5, 0},
{2, 3, 5, 6}
};
isSafe(processes, avail2, max2, alloc2);
return 0;
}
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^B Where Was


divyank_rajput@divyankrajput: ~
divyank_rajput@divyankrajput:~$ nano Banker.c
divyank_rajput@divyankrajput:~$ gcc Banker.c -o Banker
./Banker
==
CASE 1: SAFE SEQUENCE TESTING
==
[RESULT] System is NOT in a safe state! Deadlock risk detected.
==
CASE 2: UNSAFE SEQUENCE TESTING (REQUEST REJECTED)
==
[RESULT] System is NOT in a safe state! Deadlock risk detected.
divyank_rajput@divyankrajput:~$